

OPERATING SYSTEMS

BCSE303L

Dr. Naveenkumar J

Associate Professor

Dept. of Computational Intelligence

PRP 217-4

COURSE OBJECTIVES

- ❑ To introduce the operating system concepts, designs and provide skills required to implement the services.
- ❑ To describe the trade-offs between conflicting objectives in large scale system design.
- ❑ To develop the knowledge for application of the various design issues and services.

COURSE OUTCOMES

On completion of this course, student should be able to:

1. Interpret the evolution of OS functionality, structures, layers and apply various types of system calls of various process states.
2. Design scheduling algorithms to compute and compare various scheduling criteria.
3. Apply and analyze communication between inter process and synchronization techniques.
4. Implement page replacement algorithms, memory management problems and segmentation.
5. Differentiate the file systems for applying different allocation, access technique, representing virtualization and providing protection and security to OS.

SYLLABUS

Module:1	Introduction	3 hours
Introduction to OS: Functionality of OS - OS design issues - Structuring methods (monolithic, layered, modular, micro-kernel models) - Abstractions, processes, resources - Influence of security, networking, and multimedia.		
Module:2	OS Principles	4 hours
System calls, System/Application Call Interface – Protection: User/Kernel modes - Interrupts -Processes - Structures (Process Control Block, Ready List etc.), Process creation, management in Unix – Threads: User level, kernel level threads and thread models.		
Module:3	Scheduling	9 hours
Processes Scheduling - CPU Scheduling: Pre-emptive, non-pre-emptive - Multiprocessor scheduling – Deadlocks - Resource allocation and management - Deadlock handling mechanisms: prevention, avoidance, detection, recovery.		
Module:4	Concurrency	8 hours
Inter-process communication, Synchronization - Implementing synchronization primitives (Peterson's solution, Bakery algorithm, synchronization hardware) - Semaphores – Classical synchronization problems, Monitors: Solution to Dining Philosophers problem – IPC in Unix, Multiprocessors and Locking - Scalable Locks - Lock-free coordination.		

SYLLABUS

Module:5	Memory Management	7 hours
Main memory management, Memory allocation strategies, Virtual memory: Hardware support for virtual memory (caching, TLB) – Paging - Segmentation - Demand Paging - Page Faults - Page Replacement -Thrashing - Working Set.		
Module:6	Virtualization and File System Management	6 hours
Virtual Machines - Virtualization (Hardware/Software, Server, Service, Network - Hypervisors - Container virtualization - Cost of virtualization - File system interface (access methods, directory structures) - File system implementation (directory implementation, file allocation methods) - File system recovery - Journaling - Soft updates - Log-structured file system - Distributed file system.		
Module:7	Storage Management, Protection and Security	6 hours
Disk structure and attachment – Disk scheduling algorithms (seek time, rotational latency based)- System threats and security – Policy vs mechanism - Access vs authentication - System protection: Access matrix – Capability based systems - OS: performance, scaling, future directions in mobile OS.		
Module:8	Contemporary Issues	2 hours

❑ Textbook

- ❑ Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.

❑ Reference Books

- ❑ Andrew S. Tanenbaum, “Modern Operating Systems”, 2016, 4th Edition, Pearson, United Kingdom.
- ❑ William Stallings, “Operating Systems: Internals and Design Principles”, 2018, 9th Edition, Pearson, United Kingdom.

ASSESSMENTS

Assessment Head	Marks	Weightage	Submission	Tentative Date
Quiz 1	10	10	Moodle	Before CAT - 1
Continuous Assessment Test – 1 (CAT -1)	50	15	-	As per University Schedule
Quiz 2	10	10	Moodle	Before CAT - 2
Continuous Assessment Test – 1 (CAT -2)	50	15	-	As per University Schedule
Digital Assignment – 1 (handwritten)	10	10	VTOP	03/07/2023
Final Assessment Test (FAT)	100	40	-	As per University Schedule

INTRODUCTION TO OPERATING SYSTEM

- ❑ Operating System is a **System Software**.
 - ❑ System software is designed to manage the computer hardware and provide a platform for running application software.
- ❑ **Definition of Operating System**
 - ❑ It is a software that acts as an interface/intermediary between computer hardware components and the user.

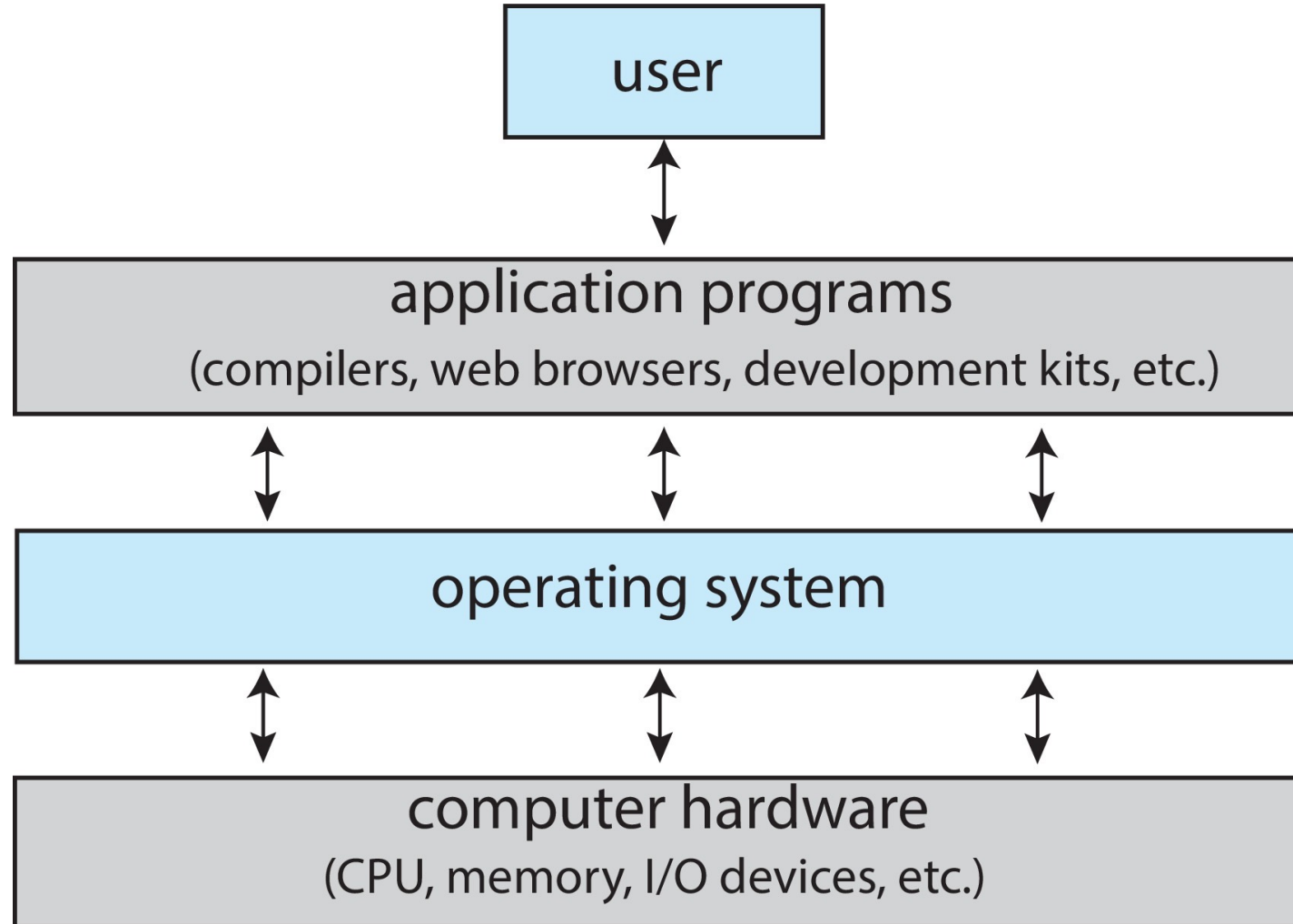
❑ **Goals of Operating System**

- ❑ Execute user programs and make solving user problems easier.
- ❑ Make the computer system convenient to use
- ❑ Use the computer hardware in an efficient manner

- ❑ The primary goal is to provide a user-friendly and convenient environment for the user.
- ❑ The secondary goal is to ensure efficient operation of the computer system by managing its resources in such a way that they are fully utilized.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Abstract View of Components of Computer



INTRODUCTION TO OPERATING SYSTEM (OS)

❑ What Operating System Do ?

❑ Operating system is a **Resource Allocator**:

❑ Manages all resources.

❑ Decides between conflicting requests for efficient and fair resource use.

❑ Operating system is a **Control Program**

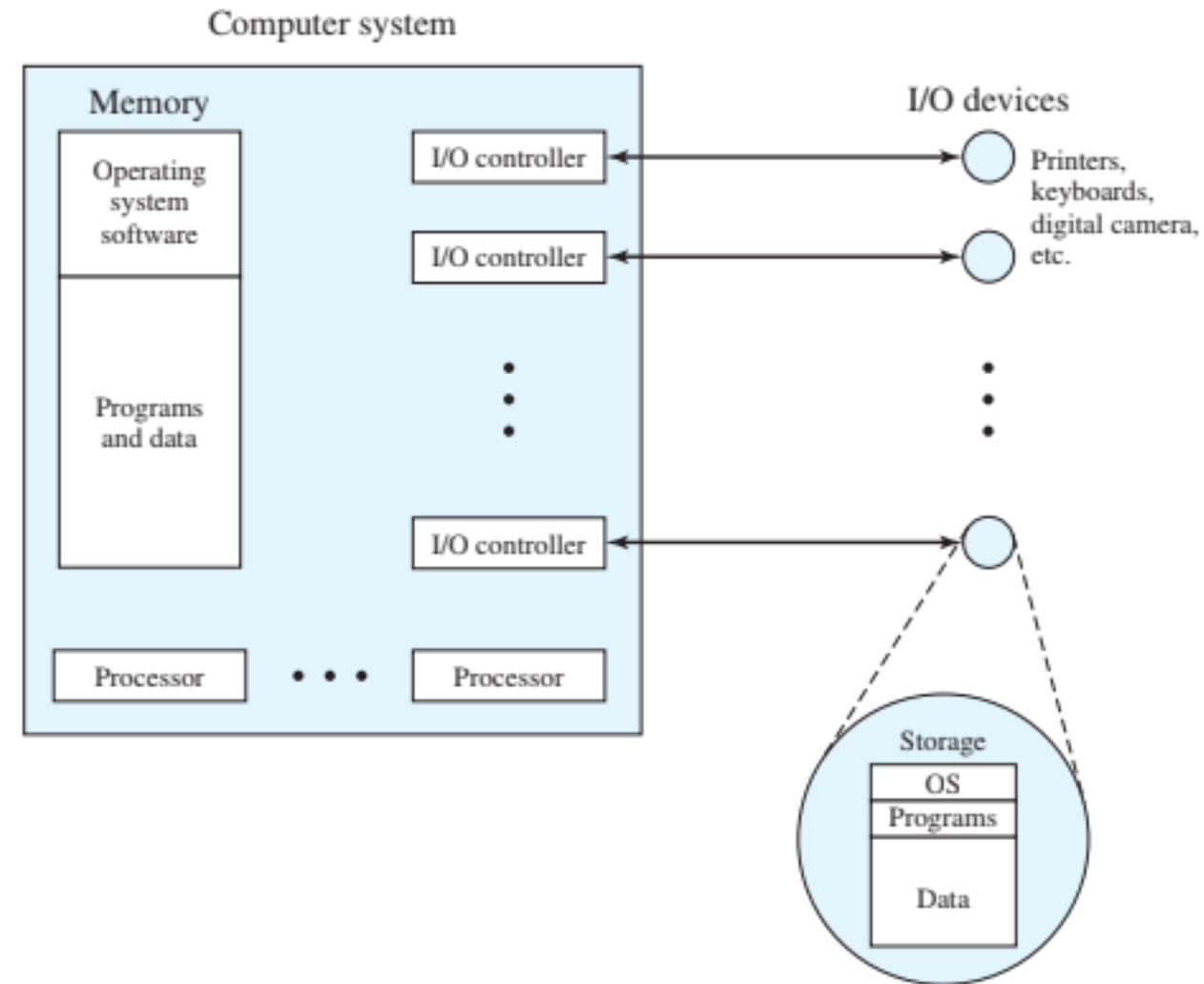
❑ Controls execution of programs to prevent errors and improper use of the computer.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ What Operating System Do ?

❑ Operating system is a **Resource Allocator**:

- ❑ Manages all resources.
- ❑ Decides between conflicting requests for efficient and fair resource use.



INTRODUCTION TO OPERATING SYSTEM (OS)

❑ What are Resources for a Operating System (Some Primary resources)

- ❑ **Central Processing Unit (CPU):** The CPU is the primary processing unit of a computer system and is responsible for executing instructions.
- ❑ **Memory:** Memory refers to the primary storage of a computer system where data and programs are stored while being processed.
- ❑ **File storage:** File storage refers to the secondary storage of a computer system where data and programs are stored permanently.
- ❑ **Input/Output (I/O) devices:** I/O devices include peripherals such as keyboards, mice, printers, and displays that allow the user to interact with the computer system.
- ❑ **Network connections:** Network connections allow the computer system to communicate with other systems and access remote resources.

❑ What Operating System Do ?

- ❑ Perform Resource Allocation.
- ❑ Control all the Programs running in their environment.
- ❑ Facilitate resource sharing.
- ❑ Facilitate optimization of programs.
- ❑ Facilitates different user interfaces.
- ❑ Makes computer systems to be used easily and with needed performance.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Components of Operating System (Primary)

- ❑ **Kernel:** The kernel is the core component of an operating system that manages the system's resources and provides low-level services to other components.
- ❑ **Process scheduler:** The process scheduler is responsible for managing the allocation of CPU time to processes and ensuring that all processes get a fair share of the CPU.
- ❑ **Memory manager:** The memory manager is responsible for managing the allocation of memory to processes and ensuring that processes do not interfere with each other's memory.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Components of Operating System (Primary)

- ❑ **File system:** The file system is responsible for organizing and managing the storage and retrieval of data on secondary storage devices such as hard disks.
- ❑ **Device drivers:** Device drivers are software components that provide an interface between the operating system and specific hardware devices such as printers, network cards, and displays.
- ❑ **User interface:** The user interface provides a way for the user to interact with the operating system and perform tasks such as launching programs and managing files.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Booting an operating system – Process

- ❑ **Power on:** When switched on the computer, the power supply sends an electrical signal to the motherboard to start the boot process.
- ❑ **BIOS/UEFI:** The motherboard's firmware, either BIOS (Basic Input/Output System) or UEFI (Unified Extensible Firmware Interface), is loaded and performs a Power-On Self Test (POST) to check that all hardware components are functioning properly.
- ❑ **Boot loader:** The boot loader is typically stored in non-volatile memory such as ROM (Read-Only Memory) or EPROM (Erasable Programmable Read-Only Memory) on the motherboard. The firmware (BIOS or UEFI) loads the boot loader from this memory and executes it. The boot loader then locates the operating system kernel on the boot device, which is typically a hard disk or solid-state drive, and loads it into memory.

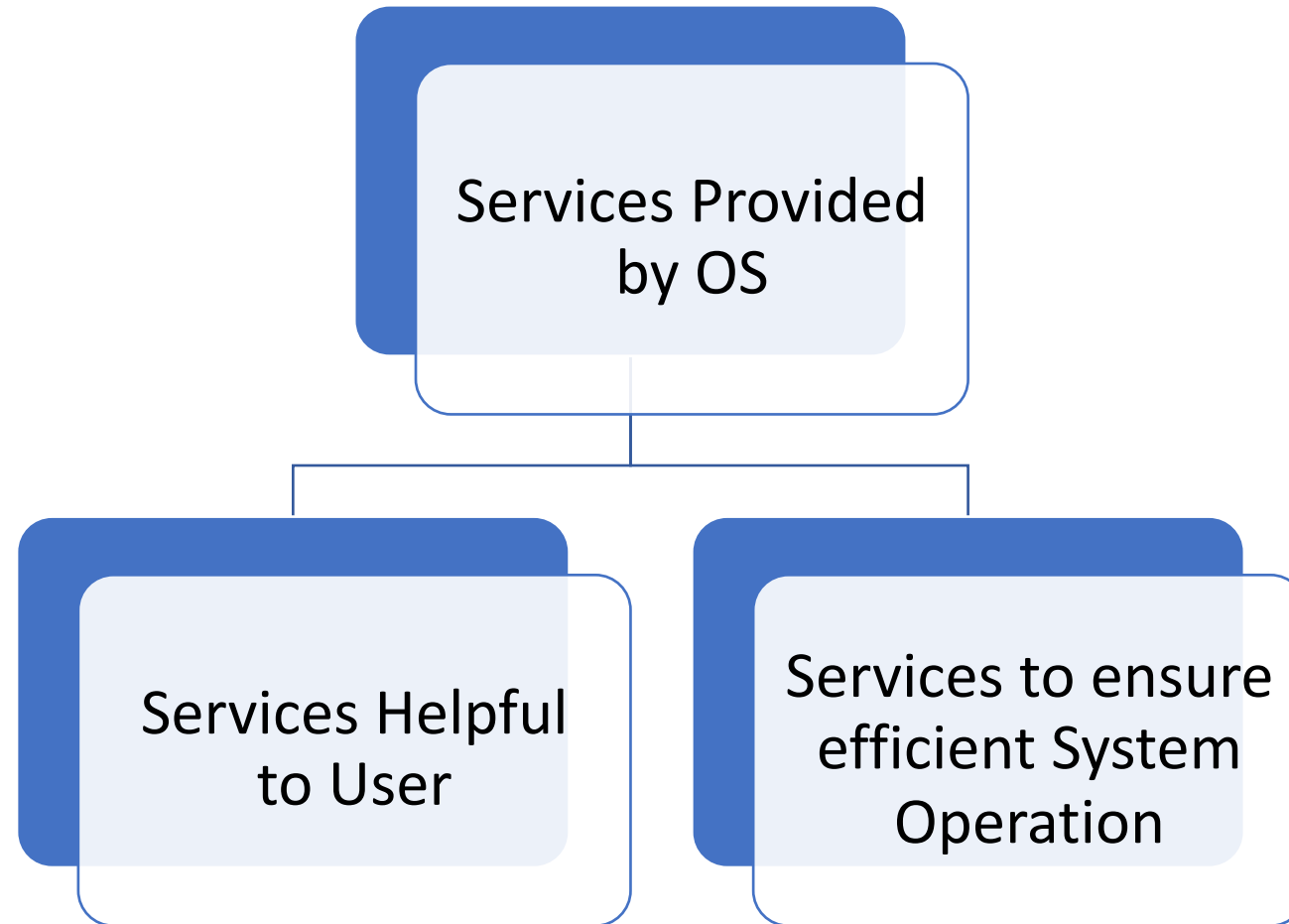
INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Booting an operating system – Process

- ❑ **Kernel:** The kernel is loaded into memory and initialized. It sets up important data structures and starts essential services such as device drivers and system daemons. This one program always running on the computer.
- ❑ **User space:** Once the kernel is initialized, it starts the user space, which includes the user interface and other system services. The user interface allows the user to log in and interact with the operating system.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ **Functionality/Services Provided by Operating System.**



INTRODUCTION TO OPERATING SYSTEM (OS)

- ❑ **Functionality/Services Provided by Operating System.** - functions that are helpful to the user:
- ❑ **User interface** - Almost all operating systems have a user interface (UI).Varies between Command-Line (CLI), Graphics User Interface (GUI), touch-screen,
- ❑ **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- ❑ **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
- ❑ **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

INTRODUCTION TO OPERATING SYSTEM (OS)

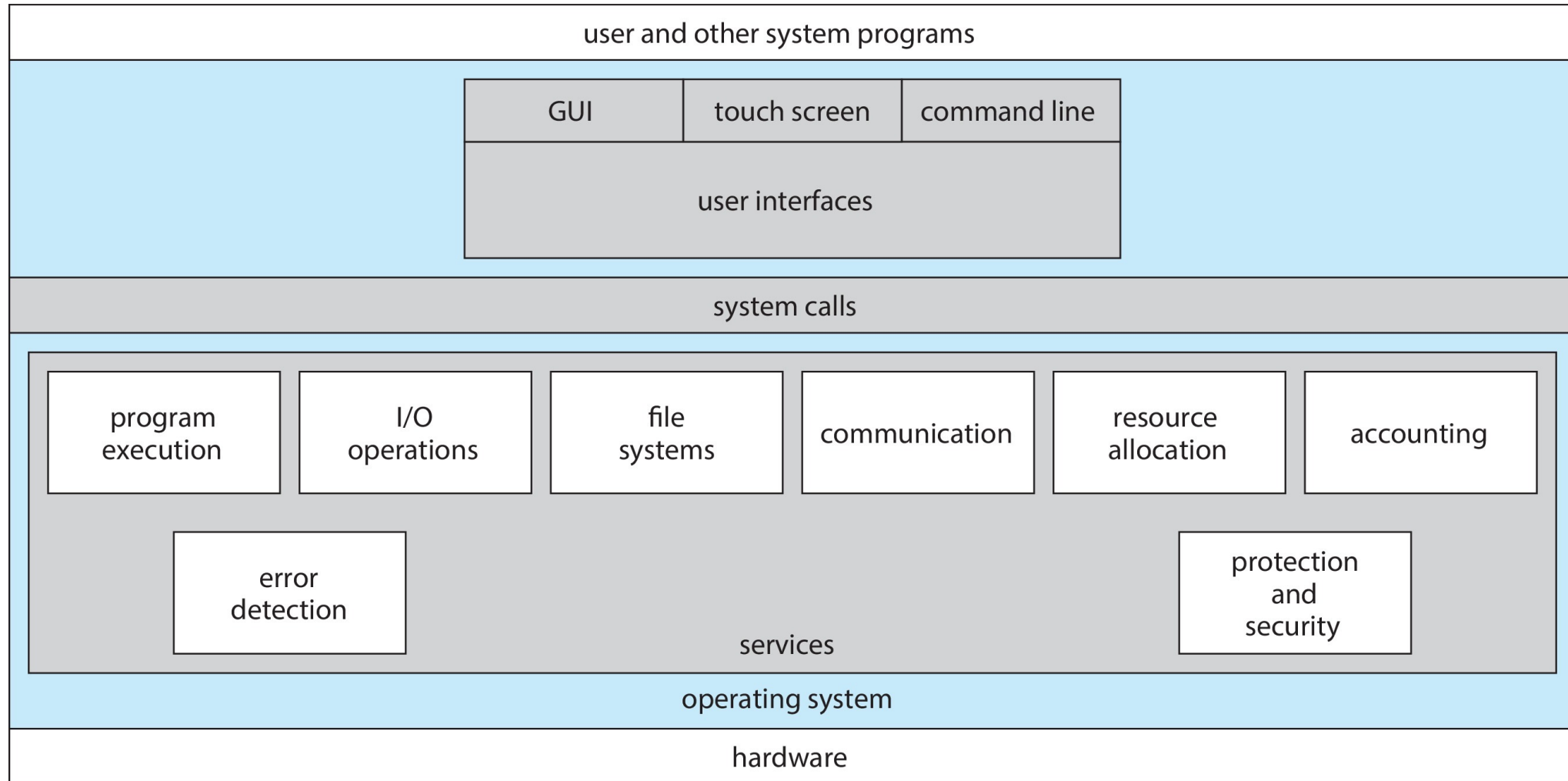
- ❑ **Functionality/Services Provided by Operating System.** - functions that are helpful to the user:
- ❑ **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - ❑ Communications may be via shared memory or through message passing (packets moved by the OS)
- ❑ **Error detection** – OS needs to be constantly aware of possible errors.
 - ❑ May occur in the CPU and memory hardware, in I/O devices, in user program
 - ❑ For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - ❑ Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system.

INTRODUCTION TO OPERATING SYSTEM (OS)

- ❑ **Functionality/Services Provided by Operating System.** - ensuring the efficient operation of the system
- ❑ **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - ❑ Many types of resources - CPU cycles, main memory, file storage, I/O devices.
- ❑ **Logging** - To keep track of which users use how much and what kinds of computer resources
- ❑ **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - ❑ **Protection** involves ensuring that all access to system resources is controlled
 - ❑ **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

INTRODUCTION TO OPERATING SYSTEM (OS)

□ **Functionality/Services View Provided by Operating System.**



INTRODUCTION TO OPERATING SYSTEM (OS)

❑ **Functioning of Operating System (Primary)**

- ❑ **Resource management:** The operating system manages and allocates memory, CPU time, and other hardware resources among the various programs and processes running on the computer.
- ❑ **Process management:** The operating system is responsible for starting, stopping, and managing processes and programs. It also controls the scheduling of processes and allocates resources to them.
- ❑ **Memory management:** The operating system manages the computer's primary memory and provides mechanisms for optimizing memory usage.
- ❑ **Security:** The operating system provides a secure environment for the user, applications, and data by implementing security policies and mechanisms such as access controls and encryption.
- ❑ **File management:** The operating system is responsible for organizing and managing the file system, including the creation, deletion, and manipulation of files and directories.
- ❑ **Device management:** The operating system manages the communication between the computer system and its peripheral devices such as printers, displays, keyboards, and mice.
- ❑ **Networking:** The operating system provides networking capabilities that allow the computer system to communicate with other systems and access remote resources.
- ❑ **User interface:** The operating system provides a user interface that allows the user to interact with the computer system and perform tasks such as launching programs and managing files.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Design Issues

- ❑ The design of the system will be affected by the **choice of hardware and the type of system**: traditional desktop/laptop, mobile, distributed, or real time.
- ❑ User Goals: The system should be convenient to use, easy to learn and to use, reliable, safe, and fast.
- ❑ Developer Requirements may be: The system should be easy to design, implement, and maintain; and it should be flexible, reliable, error free, and efficient.
- ❑ **Different requirements can result in a large variety of solutions for different environments.**

❑ Design Issues

- ❑ The separation of policy from mechanism.
 - ❑ Mechanisms determine **how to do something**; policies determine **what will be done**.
 - ❑ For example, The timer construct is a mechanism for ensuring CPU protection, but deciding how long the timer is to be set for a particular user is a policy decision.
 - ❑ Policies are likely to change across places or over time. each change in policy would require a change in the underlying mechanism.

❑ Design Issues

- ❑ The separation of policy from mechanism.
 - ❑ Mechanisms determine **how to do something**; policies determine **what will be done**.
 - ❑ For example, The timer construct is a mechanism for ensuring CPU protection, but deciding how long the timer is to be set for a particular user is a policy decision.
 - ❑ **Policies are likely to change across places or over time.** each change in policy would require a change in the underlying mechanism.

❑ Design Issues

- ❑ Once an operating system is designed, it must be implemented.
- ❑ Operating systems are collections of many programs, written by many people over a long period of time, it is difficult to make general statements about how they are implemented.
- ❑ Languages used – C, C++, assembly language,
- ❑ an operating system is far easier to port to other hardware if it is written in a higher-level language

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Structuring Operating system

❑ Monolithic Structure

- ❑ Place all of the functionality of the kernel into a single, static binary file that runs in a single address space.
- ❑ Example - the original UNIX operating system, which consists of two separable parts: the kernel and the system programs.
- ❑ The kernel is further separated into a series of interfaces and device drivers
- ❑ The kernel provides the file system, CPU scheduling, memory management, and other operating- system functions through system calls.

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Structuring Operating system - **Monolithic Structure**

- ❑ The entire operating system is working in kernel space
- ❑ All code runs in kernel space, with no separation between kernel and user-level processes.
- ❑ The kernel is designed to provide all operating system services, including memory management, process scheduling, device drivers, and file systems, in a single, large binary file.
- ❑ Merits:
 - ❑ It can provide high performance since system calls can be made directly to the kernel without the overhead
 - ❑ The design is simpler.

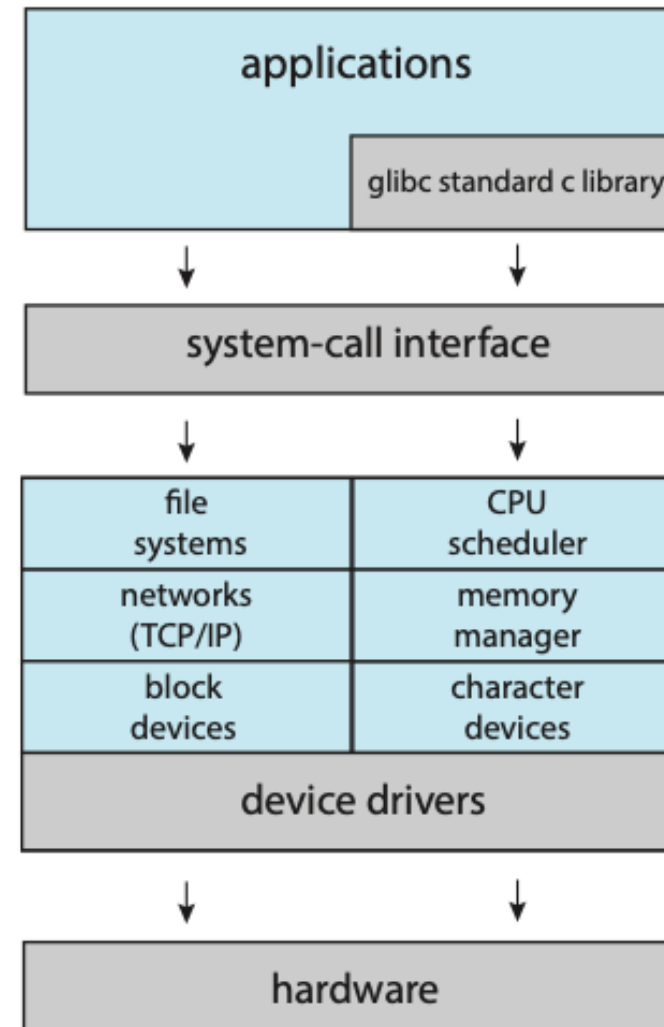
❑ Structuring Operating system - **Monolithic Structure**

❑ Demerits

- ❑ It can lead to a less secure and less stable operating system.
- ❑ Since all code runs in kernel space, any vulnerabilities or bugs in the kernel can potentially affect the entire system.
- ❑ If a user-level process crashes, it can bring down the entire system since there is no separation between kernel and user-level processes.

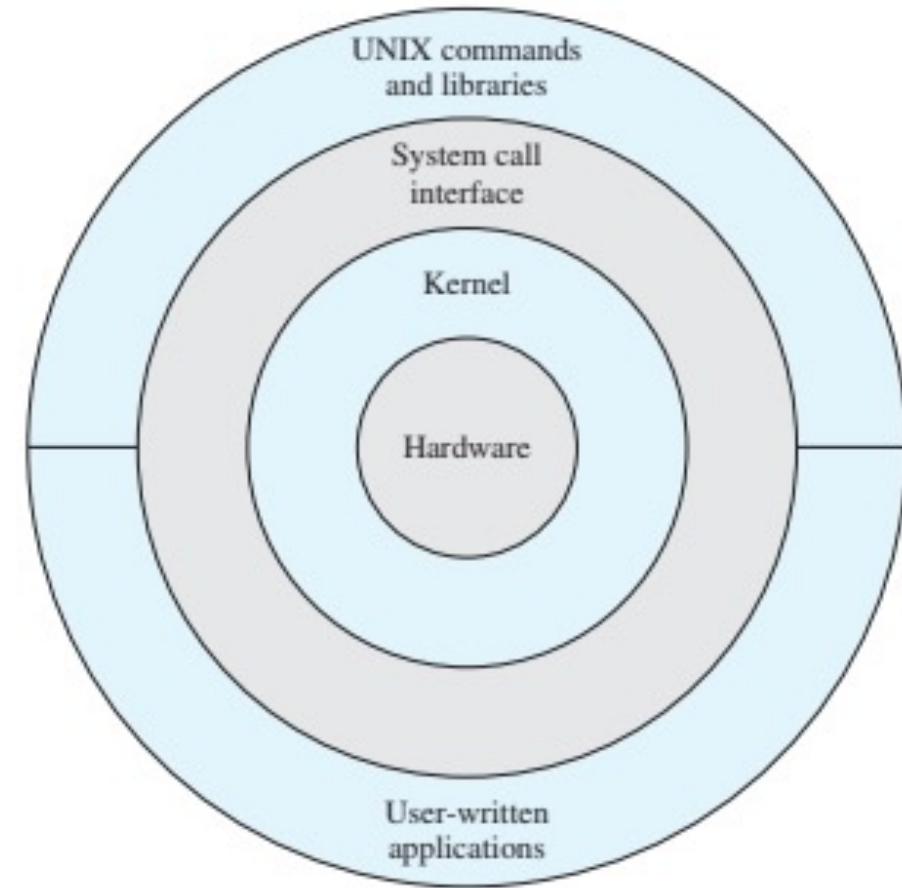
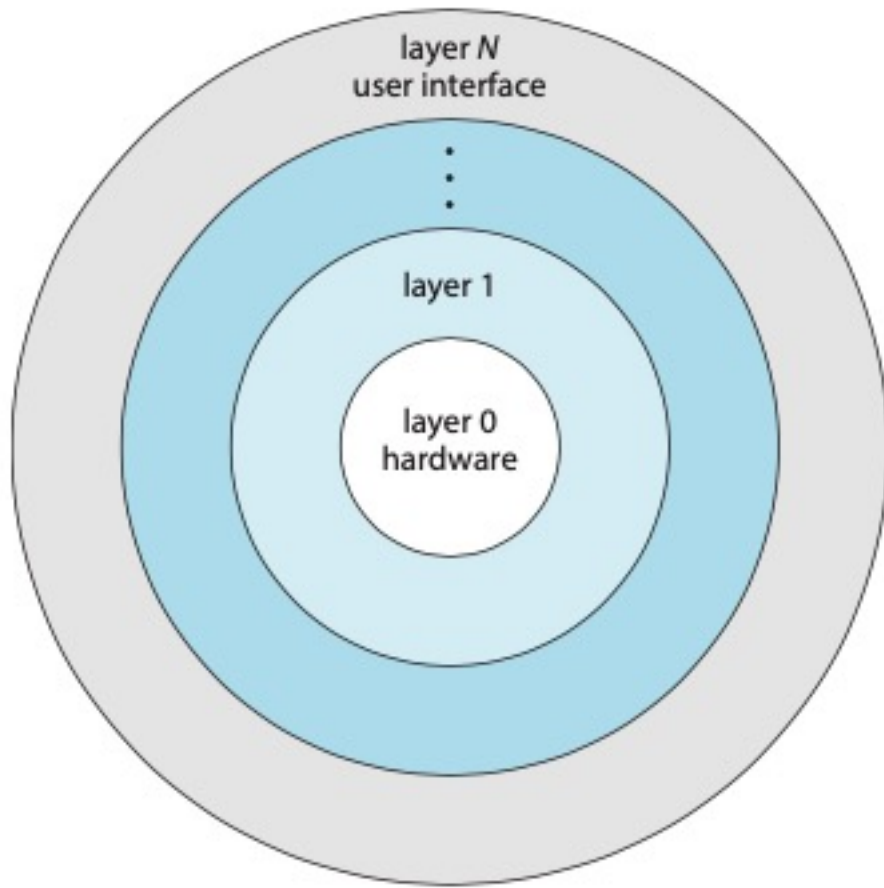
INTRODUCTION TO OPERATING SYSTEM (OS)

□ Structuring Operating system - Monolithic Structure – Example Linux



INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Structuring Operating system – Layered Approach



INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Structuring Operating system – Layered (Modular) Approach

- ❑ System is divided into separate, smaller components that have specific and limited functionality. All these components together comprise the kernel.
- ❑ The operating system is broken into several layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.
- ❑ The different services of the operating system are split into various layers. Each layer is responsible for performing a specific function and can only access the services provided by the layers below it.

❑ Structuring Operating system – Layered (Modular) Approach

❑ Merit:

- ❑ Changes in one component affect only that component, and no others.
- ❑ Simplicity of construction and debugging, making it easier to develop, test, and maintain.

❑ DeMerit:

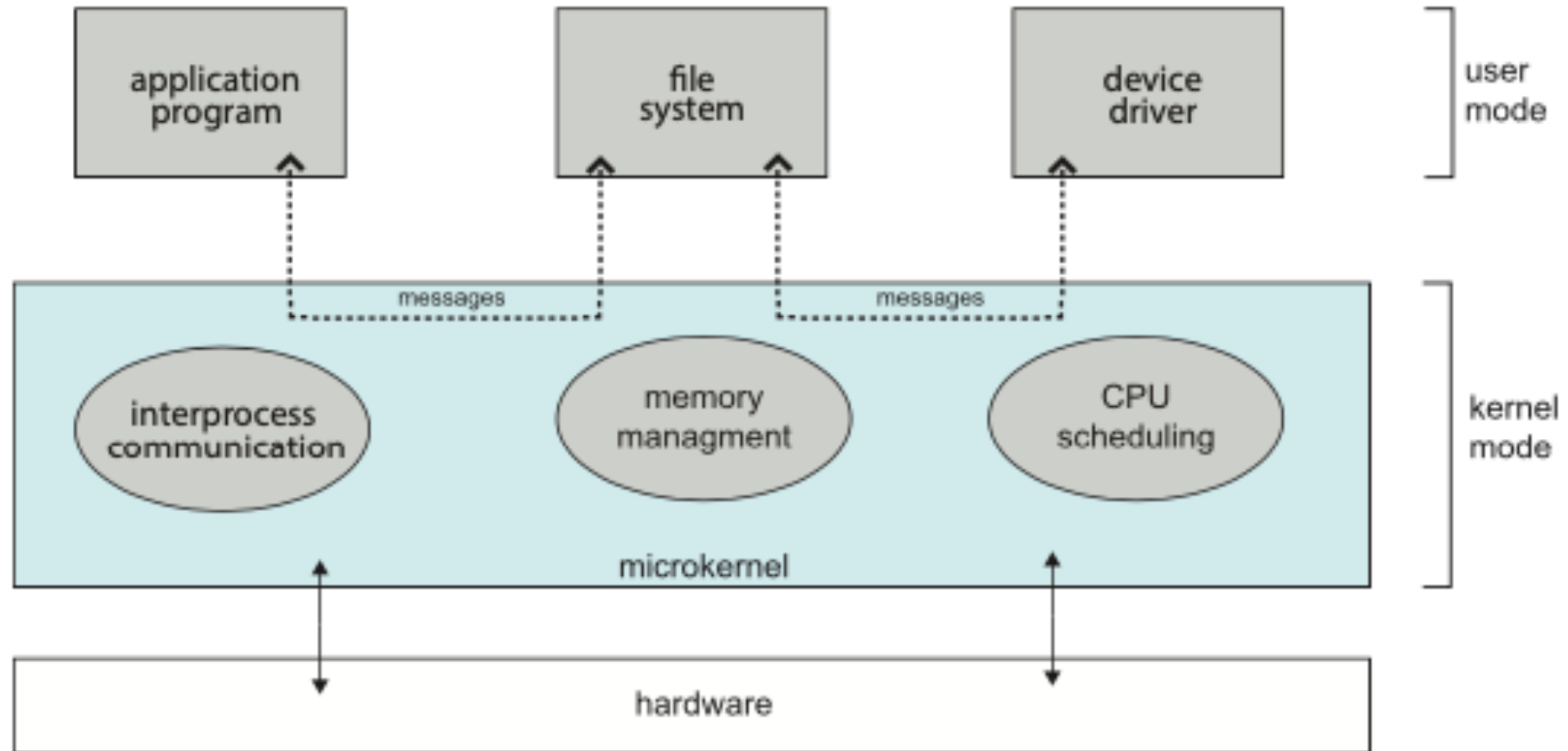
- ❑ The challenges of appropriately defining the functionality of each layer.
- ❑ The overall performance of such systems is poor due to the overhead of requiring a user program to traverse through multiple layers to obtain an operating-system service.

❑ Structuring Operating system – Microkernels Approach

- ❑ Structures the operating system by removing all nonessential components from the kernel and implementing them as user- level programs that reside in separate address spaces.
- ❑ The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space.
- ❑ Communication is provided through message passing

INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Structuring Operating system – Microkernels Approach



INTRODUCTION TO OPERATING SYSTEM (OS)

❑ Structuring Operating system – **Microkernels Approach**

❑ **Merits:**

- ❑ It makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel.
- ❑ The resulting operating system is easier to port from one hardware design to another.
- ❑ Provides more security and reliability, since most services are running as user

❑ **Demerits:**

- ❑ The performance of microkernels can suffer due to increased system-function overhead.
- ❑ Operating system may have to switch from one process to the next to exchange the messages. The overhead involved in copying messages and switching between processes has been the largest impediment to the growth of microkernel-based operating systems.

❑ Structuring Operating system – Loadable Kernel Modules Approach

- ❑ The kernel has a set of core components and can link in additional services via modules, either at boot time or during run time.
- ❑ A Loadable Kernel Module (LKM) is an object file that contains code to extend the running kernel, or so-called base kernel, of an operating system.
- ❑ Used to add support for new hardware (as device drivers), file systems, or system calls.
- ❑ The idea of the design is for the kernel to provide core services, while other services are implemented dynamically, as the kernel is running.

❑ Structuring Operating system – Loadable Kernel Modules Approach

❑ When the functionality provided by an LKM is no longer required, it can be unloaded in order to free memory and other resources.

❑ Merits:

❑ Allows for greater flexibility and modularity in the design of an operating system

❑ functionality can be loaded and unloaded as and when required.

❑ **Abstractions**

- ❑ An operating system transforms the physical world of devices, instructions, memory, and time into a virtual world that is the result of abstractions built by the operating system.
- ❑ The most fundamental abstractions in operating systems is the “process

❑ The Process

- ❑ The concept of process is fundamental to the structure of operating systems.
- ❑ A program in execution.
- ❑ An instance of a program running on a computer.
- ❑ The entity that can be assigned to and executed on a processor.
- ❑ A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources.

❑ Influence of Security

Security and protection as it relates to operating systems can be roughly grouped into four categories:

- ❑ **Availability:** Concerned with protecting the system against interruption
- ❑ **Confidentiality:** Assures that users cannot read data for which access is unauthorized
- ❑ **Data integrity:** Protection of data from unauthorized modification
- ❑ **Authenticity:** Concerned with the proper verification of the identity of users and the validity of messages or data.